

Git, Github e Introdução ao GitFlow

Engenharia de Qualidade e Confiabilidade

Prof. Me. Sergio Souza Novak — UniRV

11 de junho de 2026



- 1 Fundamentos: O que é o Git?
- 2 Git vs GitHub
- 3 Introdução ao GitFlow e Problemas
- 4 Conceito de Git Flow
- 5 Vantagens e Desvantagens
- 6 Unidade 2 – Estrutura do Git Flow
- 7 Fluxo de Trabalho Detalhado

O que é o Git?

- O Git é um **Sistema de Controle de Versão Distribuído** (DVCS).
- Ferramenta essencial para o desenvolvimento de software moderno.
- Permite rastrear mudanças em arquivos e coordenar o trabalho entre várias pessoas.
- É de código aberto (*Open Source*) e totalmente gratuito.

- Criado em **2005** por **Linus Torvalds** (o mesmo criador do Linux).
- Motivação: A comunidade de desenvolvimento do Kernel Linux precisava de uma ferramenta rápida, descentralizada e que suportasse fluxos de trabalho não lineares (milhares de desenvolvedores independentes).
- "Git" é uma gíria britânica para "pessoa teimosa ou ignorante" (Linus brincou que sempre nomeia os projetos com seu próprio nome).

Por que usar Controle de Versão?

Sem controle de versão, lidamos com arquivos assim:

```
projeto_final.doc  
projeto_final_v2.doc  
projeto_final_agora_vai.doc  
projeto_final_sergio_revisado.doc
```

Com o Git:

- Todo o histórico fica encapsulado na pasta do projeto.
- Se o projeto quebrar, é possível voltar no tempo para qualquer versão anterior.
- Fica registrado **quem** modificou o quê, **quando** e **por quê**.

Sistemas Centralizados (SVN, CVS)

- Existe apenas *um* servidor central com o histórico.
- Se o servidor cair, ninguém consegue versionar o código.
- Operações dependem de internet.

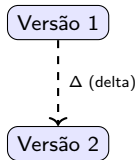
Sistemas Distribuídos (Git)

- Cada desenvolvedor tem um "clone" completo do histórico na própria máquina.
- Se o servidor explodir, qualquer máquina local serve como backup total.
- Maioria das operações é local e instantânea.

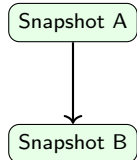
Como o Git pensa: Snapshots

- Outros sistemas (SVN) armazenam alterações baseadas em arquivo (**deltas**).
- O Git pensa em seus dados como uma série de "fotografias"(**snapshots**).
- Ao salvar o estado, o Git tira uma foto do sistema de arquivos e armazena uma referência.

SVN (Deltas)



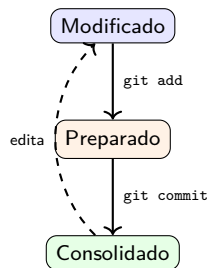
Git (Snapshots)



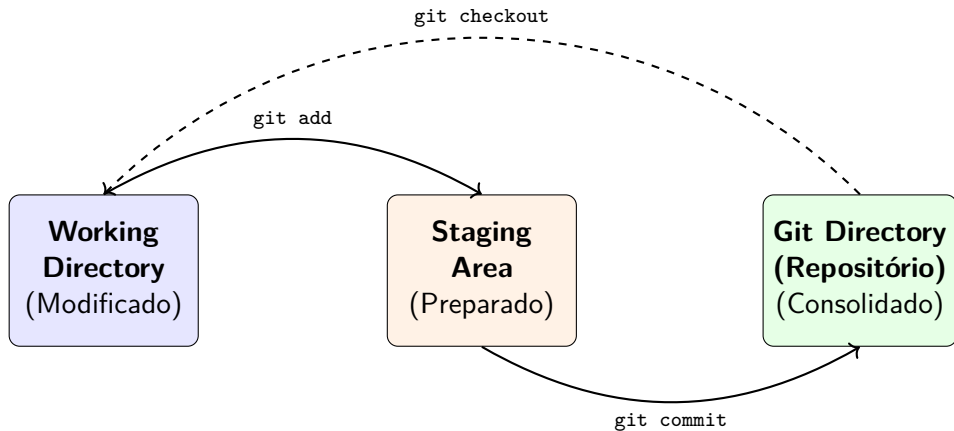
Os Três Estados do Git

Arquivos no Git podem residir em três estados:

- 1 **Modificado**: O arquivo foi alterado, mas ainda não foi salvo.
- 2 **Preparado (*Staged*)**: Marcado para ir no próximo commit.
- 3 **Consolidado (*Committed*)**: Armazenado de forma segura.



Áreas de Trabalho do Git



Por que o Git dominou o mercado?

- **Velocidade:** Operações locais não dependem de rede.
- **Design simples:** Embora poderoso, a estrutura de dados interna é elegante.
- **Suporte forte a branches:** Criar e mesclar branches no Git leva frações de segundos, o que encoraja fluxos de trabalho baseados em ramificações (como o próprio GitFlow).
- **Integridade Criptográfica:** Tudo no Git gera um hash SHA-1. É impossível alterar um arquivo, data ou autor sem que o Git saiba.

Conceitos Essenciais: Repositório e Commit

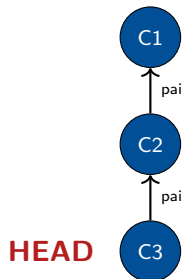
Repositório (Repo)

Pasta rastreada pelo Git. Contém o diretório oculto `.git`.

Commit

Uma versão do projeto (Snapshot). Contém:

- Alterações (arquivos).
- Autor, data e mensagem.
- Referência ao commit **anterior** (corrente).



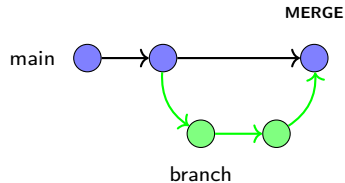
Conceitos Essenciais: Branch e Merge

Branch (Ramificação)

Linha do tempo paralela e independente.

Merge (Mesclagem)

Processo de juntar o histórico bifurcado de volta.



Cenário Prático: Sistema de Lanchonete

- Para entender os comandos na prática, vamos imaginar o desenvolvimento de um **Sistema de Lanchonete**.
- Nosso projeto começa com dois arquivos principais em JavaScript:
 - `cardapio.js`: Gerencia os itens à venda.
 - `pedidos.js`: Gerencia a fila de pedidos dos clientes.
- Vamos acompanhar o fluxo de trabalho do Git passo a passo para evoluir essa lanchonete.

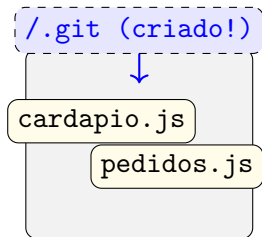
`cardapio.js`
(vazio)

`pedidos.js`
(vazio)

git init – Inaugurando a Lanchonete

- O comando `git init` transforma uma pasta comum em um repositório Git.
- Ele cria a pasta oculta `.git`, onde todo o "cérebro" e histórico do projeto serão armazenados.

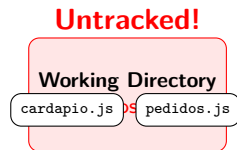
```
$ git init
Initialized empty Git repository
in /lanchonete/.git/
```



git status – Inspeccionando a Cozinha

- O comando `git status` mostra o estado atual da sua "cozinha".
- Ele revela arquivos modificados que ainda não foram preparados (*Untracked* ou *Modified*).

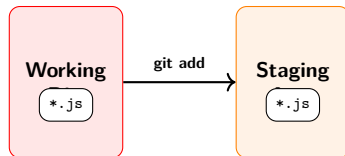
```
$ git status
On branch main
Untracked files:
  (use "git add <file>..." to include)
       cardapio.js
       pedidos.js
```



git add – Preparando os Ingredientes

- O comando `git add` move os arquivos para a *Staging Area* (Área de Preparação).
- É como separar os ingredientes na bancada antes de finalizar o prato.

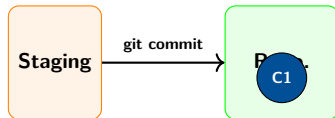
```
$ git add cardapio.js pedidos.js  
# ou git add . para adicionar tudo
```



git commit – Fechando o Pedido

- O comando `git commit` tira a "foto" (*snapshot*) do que está na *Staging Area* e salva no repositório.
- Sempre acompanhado de uma mensagem clara (`-m`).

```
$ git commit -m "feat: cria estrutura inicial"  
[main (root-commit) a1b2c3d] feat: cria estrutura inicial  
2 files changed, 50 insertions(+)
```



git log – Histórico de Operações

- O comando `git log` exibe o diário da lanchonete: quem fez o quê e quando.
- Mostra o autor, data, hash do commit e a mensagem.

```
$ git log --oneline  
a1b2c3d (HEAD -> main) feat: cria estrutura inicial
```

Commit a1b2c3d...

Autor: Sergio <sergio@unirv.br>

Data: Hoje

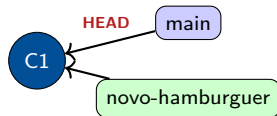
feat: cria estrutura inicial



git branch – Criando um Menu Alternativo

- Queremos testar um novo hambúrguer sem estragar o cardápio oficial.
- O comando `git branch` cria uma linha do tempo paralela.

```
$ git branch novo-hamburguer
$ git branch
* main
  novo-hamburguer
```

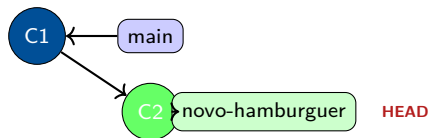


git checkout – Trocando de Bancada

- Criar a branch não muda para ela automaticamente. Usamos `git checkout` (ou `git switch`) para "pular" para a nova linha do tempo.

```
$ git checkout novo-hamburguer
Switched to branch 'novo-hamburguer'

# Adicionando o sanduiche no cardapio.js
$ git add cardapio.js
$ git commit -m "feat: lanca o x-bacon"
```

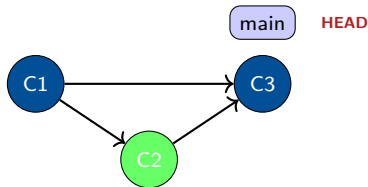


git merge – Incorporando ao Menu Oficial

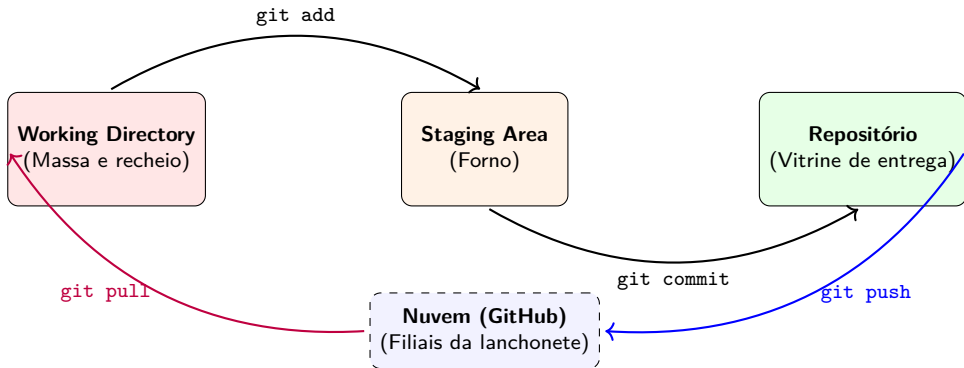
- O X-Bacon foi um sucesso! Agora precisamos juntá-lo ao cardápio principal.
- Voltamos para a main e usamos `git merge`.

```
$ git checkout main
Switched to branch 'main'

$ git merge novo-hamburguer
Merge made by the 'ort' strategy.
 cardapio.js | 10 ++++++++
 1 file changed, 10 insertions(+)
```



Resumo Visual do Fluxo da Lanchonete



- **git push:** Envia os commits locais para a nuvem.
- **git pull:** Traz as novidades da nuvem para a sua lanchonete.

O que é o GitHub?

- O GitHub é uma **plataforma de hospedagem** de código-fonte em nuvem que utiliza o Git.
- Adiciona funcionalidades de rede social ao desenvolvimento (seguir projetos, estrelas, issues, pull requests).
- Foi comprado pela Microsoft em 2018 e é o maior repositório de código do planeta.
- Existem alternativas concorrentes que também usam Git: **GitLab**, **Bitbucket**, **Gitea**.

Git

- É a **ferramenta**.
- Software de linha de comando instalado no seu computador.
- Trabalha localmente e *offline*.
- Não possui interface gráfica nativa (apenas terminal).

GitHub

- É o **serviço** web.
- Site na internet hospedando repositórios Git.
- Trabalha na nuvem (*online*).
- Oferece interface gráfica rica, gestão de projetos, CI/CD.

Problemas no Desenvolvimento Colaborativo (O Caos)

Antes de adotar um fluxo estruturado, as equipes sofrem com:

- **Código Quebrado:** Funcionalidades inacabadas vão parar no cliente acidentalmente.
- **Conflitos Constantes:** Desenvolvedores alterando os mesmos arquivos na main ao mesmo tempo.
- **Sem Rastreabilidade:** "Quem subiu esse código? Qual versão corrigiu aquele problema?"

```
$ git push origin main
To https://github.com/empresa/projeto.git
! [rejected] main -> main (fetch first)
error: failed to push some refs
hint: Updates were rejected because the
remote contains work that you do not have
locally.
```

```
$ git pull origin main
Auto-merging index.html
CONFLICT (content): Merge conflict in
index.html
Automatic merge failed; fix conflicts and
then commit the result.
```

A Solução: Organização de Versões de Software

Para evitar o caos, precisamos adotar o **Isolamento** de ambientes (Dev, Teste, Produção) e **Padronização**.

Versionamento Semântico (SemVer)

Padroniza como os números de versão são definidos para comunicar o nível de impacto das mudanças.

Exemplo: v2.1.4

MAJOR (2)
Mudança
Incompatível

▪

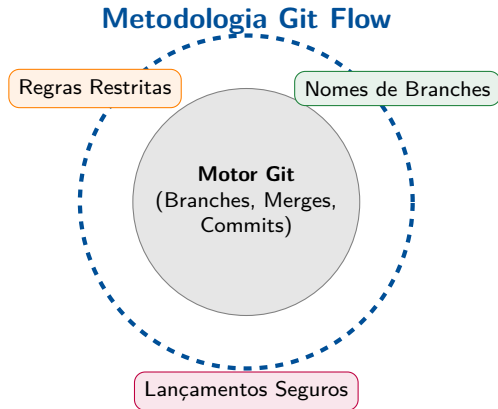
MINOR (1)
Nova Função
(Compatível)

▪

PATCH (4)
Correção de Bug
(Compatível)

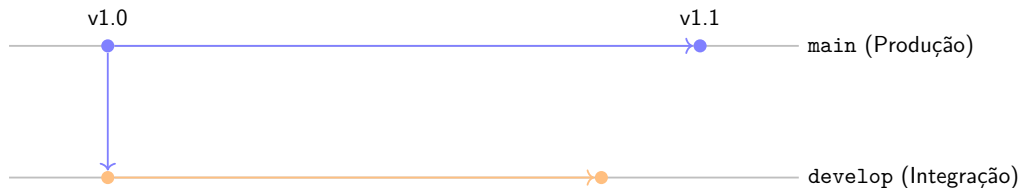
Conceito de Git Flow

- Criado por **Vincent Driessen** (2010).
- Não é um comando nativo ou plugin obrigatório, mas sim um **conjunto estrito de regras** e convenções.
- **Objetivo:** Fornecer um modelo de ramificação rigoroso para organizar projetos em larga escala.
- Excelente para produtos de software tradicionais com **cronogramas de lançamento** definidos.



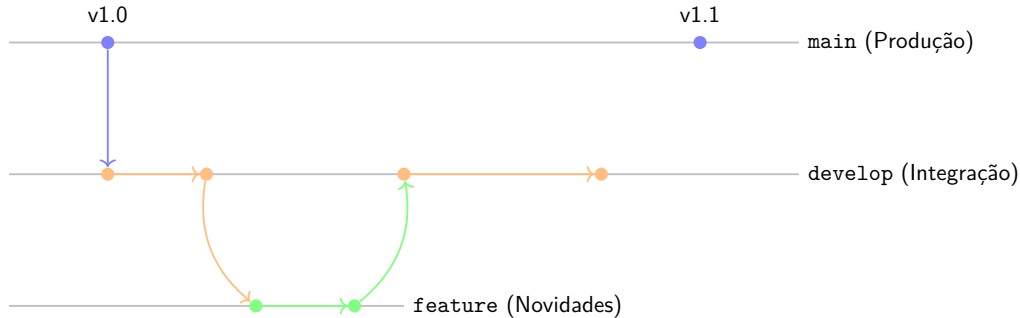
Visão Geral do Fluxo (1. Branches Principais)

- **main:** Armazena o código de produção estável.
- **develop:** Base para a próxima versão, onde o código em desenvolvimento é integrado.



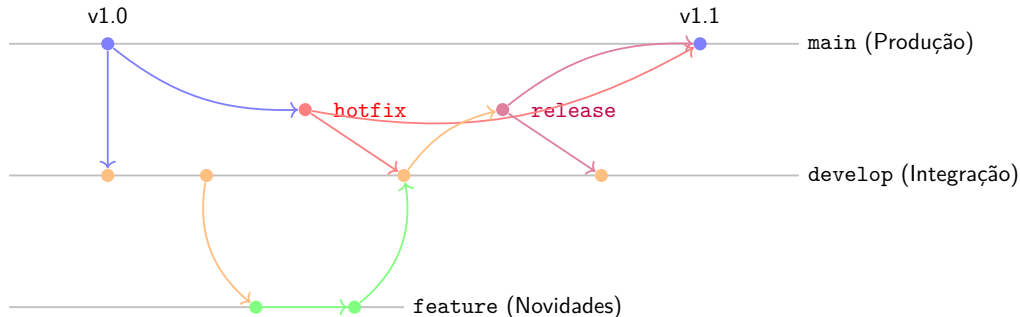
Visão Geral do Fluxo (2. Feature Branches)

- **feature:** Ramificações criadas a partir da `develop` para implementar novas funcionalidades. Retornam para a `develop`.



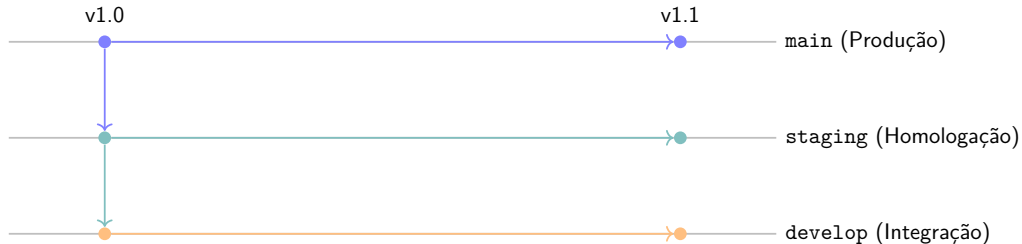
Visão Geral do Fluxo (3. Release e Hotfix)

- **release:** Prepara o lançamento. Ao finalizar, vai para main e develop.
- **hotfix:** Corrige erros críticos direto em produção (a partir da main).



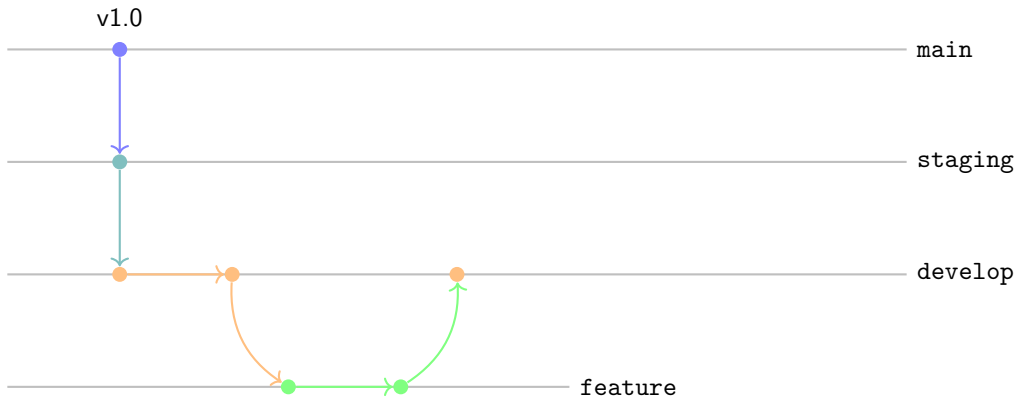
Visão com Staging (1. Estrutura Base)

- Adicionamos a branch **staging** entre **main** e **develop**.
- Usada para homologação (QA) em um ambiente idêntico à produção.



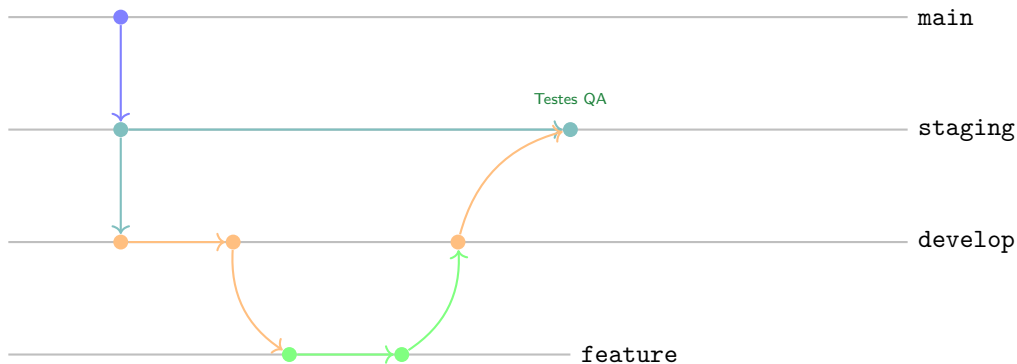
Visão com Staging (2. Desenvolvimento Local)

- O ciclo de desenvolvimento (feature) continua o mesmo: surge da develop e retorna para ela.



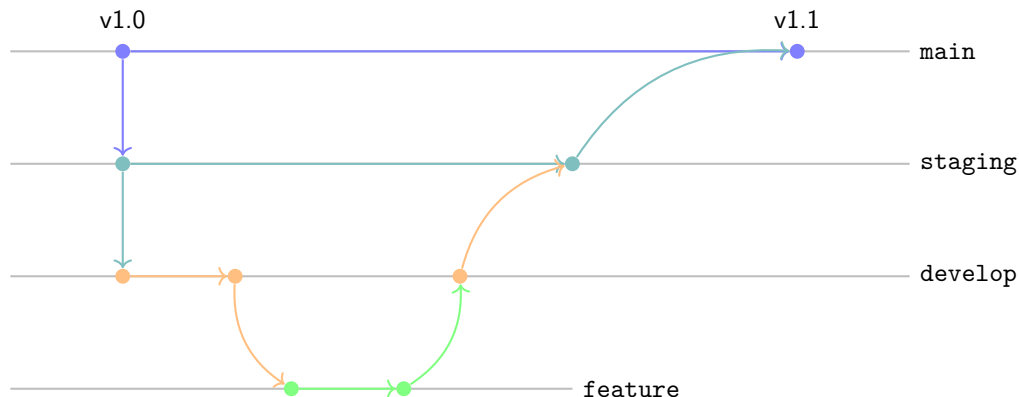
Visão com Staging (3. Promovendo para Homologação)

- Quando as funcionalidades estão maduras, a develop é enviada para a staging.
- Na staging, a equipe de Qualidade (QA) realiza testes exhaustivos.



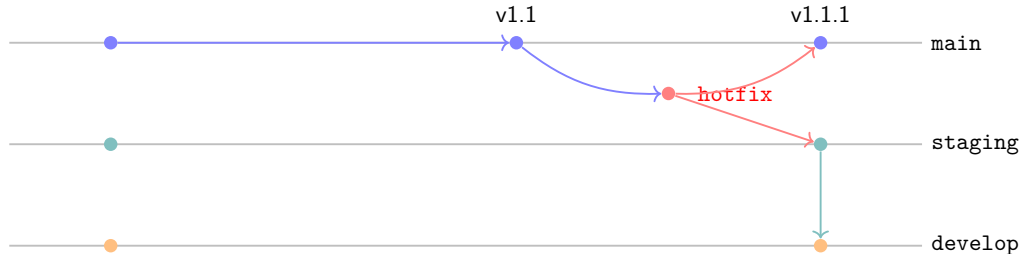
Visão com Staging (4. Lançamento em Produção)

- Após aprovada na staging, essa mesma versão é mesclada na main e taguada.
- Isso garante que exatamente o código que foi homologado irá para produção.



Visão com Staging (5. Tratando Hotfixes)

- O hotfix nasce da main para corrigir bugs críticos em produção.
- Ao finalizar, ele deve ser mesclado na main, e também na staging e na develop.



Vantagens e Desvantagens

Vantagens (+)

- Histórico de projeto limpo e rastreável.
- Separação clara entre desenvolvimento, testes e produção.
- Facilita o trabalho paralelo em grandes equipes.
- Suporta múltiplas versões ativas do software simultaneamente.

Desvantagens (-)

- Histórico denso com muitos *merge commits*.
- Curva de aprendizado moderada para novos desenvolvedores.
- Complexidade desnecessária para projetos pequenos ou startups.
- Pode conflitar com estratégias de Entrega Contínua (Continuous Deployment).

A estrutura do Git Flow categoriza os branches com base em seu **tempo de vida** e **propósito**.

- **Branches Principais (Longa duração):**

- `main` (ou `master`): Código em produção, altamente estável.
- `develop`: Base para o próximo release, onde o código é integrado.
- `staging`: (Opcional, mas comum) Ambiente de homologação/pré-produção.

- **Branches Auxiliares (Curta duração):**

- `feature/*`: Desenvolvimento de novas funcionalidades.
- `release/*`: Preparação de uma versão para lançamento.
- `hotfix/*`: Correções de emergência em produção.

Branches Principais: main e develop

- A main só recebe merges de release ou hotfix. Cada commit na main geralmente recebe uma Tag (ex: v1.2.0).
- A develop contém as últimas alterações entregues para a próxima versão.

Exemplo de inicialização:

```
# Inicializa o repositório  
git init  
  
# O primeiro commit vai para a main  
git commit --allow-empty -m "Commit inicial"  
  
# Criando a branch develop a partir da main  
git branch develop  
git checkout develop
```

Iniciando o GitFlow automaticamente (git flow init)

- O comando `git flow init` inicializa um repositório Git em uma pasta e configura toda a estrutura de ramificações automaticamente.
- Ele fará uma série de perguntas sobre como nomear as ramificações. O padrão é pressionar Enter para todas.

```
$ git flow init
No branches exist yet. Base branches must be created now.
Branch name for production releases: [main]
Branch name for "next release" development: [develop]

How to name your supporting branch prefixes?
Feature branches? [feature/]
Release branches? [release/]
Hotfix branches? [hotfix/]
```

A Branch staging (Homologação)

- Embora o modelo original de Vincent Driessen não a cite explicitamente, a branch staging é amplamente usada em cenários corporativos reais.
- Atua como uma ponte entre a develop/release e a main.
- **Propósito:** Implantar o código em um ambiente espelho da produção para testes finais de QA (Quality Assurance) e aceite do cliente.

```
# Levando o código da develop para staging para testes  
git checkout staging  
git merge develop --no-ff  
git push origin staging
```

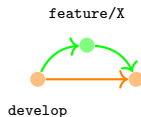
Branches Auxiliares: feature

- **Origem:** develop | **Destino:** develop
- Usadas para desenvolver novas funcionalidades.

```
# Criando a feature
git checkout -b feature/login develop

# Trabalhando e "commitando"
git commit -am "Adiciona tela de login"

# Finalizando a feature (merge em develop)
git checkout develop
git merge --no-ff feature/login
git branch -d feature/login
```



Branches Auxiliares: release e hotfix

- **release/*:** (Origem: develop) Para estabilizar a próxima versão (apenas *bugfixes*). Finaliza em main e develop.
- **hotfix/*:** (Origem: main) Correção de emergência em produção. Finaliza em main e develop.

```
# Criando um hotfix a partir da main
git checkout -b hotfix/v1.0.1 main
git commit -am "Corrige bug critico"

# Finalizando (merge em main e develop)
git checkout main
git merge --no-ff hotfix/v1.0.1
git tag -a v1.0.1 -m "Versao 1.0.1"

git checkout develop
git merge --no-ff hotfix/v1.0.1
```

1. Ciclo de Vida no GitFlow

O **fluxo de trabalho** do GitFlow descreve o ciclo de vida de uma alteração no código, desde a concepção até a chegada em produção.

- 1 **Desenvolvimento de funcionalidades** (*features*).
- 2 **Integração do código** (*merge* na *develop*).
- 3 **Preparação de versões** (*releases*).
- 4 **Correções emergenciais** (*hotfixes*).
- 5 **Versionamento** (*tags* semânticas).

2. Desenvolvimento de Funcionalidades

- Toda nova ideia, requisito ou alteração técnica começa com uma nova branch de **feature**.
- **Regra de ouro: NUNCA** desenvolva ou comite diretamente na `develop` ou na `main`.
- O isolamento permite que dezenas de desenvolvedores trabalhem em tarefas diferentes sem gerar conflitos diários ou quebrar o código dos colegas.

3. Criando uma Feature Localmente

A branch de feature **sempre** nasce a partir da develop.

```
# 1. Garanta que voce esta na develop
git checkout develop

# 2. Puxe as ultimas alteracoes do servidor (boa pratica)
git pull origin develop

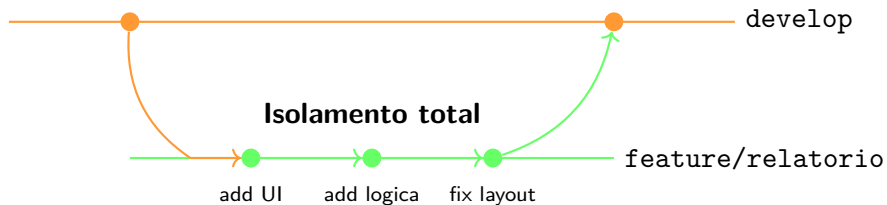
# 3. Crie e mude para a nova branch de feature
# O prefixo 'feature/' ajuda na organizacao visual
git checkout -b feature/relatorio-vendas
```

4. Trabalhando na Feature

Boas Práticas durante o desenvolvimento da Feature:

- **Commits pequenos e atômicos:** Faça commits lógicos (ex: "Cria botão X", "Adiciona rota Y") em vez de um gigantesco no final do dia.
- **Mensagens claras:** A mensagem do commit deve explicar o *porquê* da mudança.
- **Sincronização:** Se o projeto for muito ativo e a `develop` mudar muito enquanto você desenvolve, faça *merge* (ou *rebase*) da `develop` para dentro da sua feature frequentemente.

5. O Ciclo da Feature (Diagrama)



6. Integração na Branch develop

- Quando a feature está pronta, testada e completa, ela precisa voltar para o código comum (develop).
- Essa integração (*merge*) precisa ser feita com cuidado para evitar quebrar a compilação global do projeto.
- A abordagem de integração depende se a equipe usa revisão de código centralizada ou trabalho local puramente colaborativo.

7. Pull Requests (PR) vs Merge Local

Pull Request (Github/Gitlab)

- A feature é empurrada (*push*) para o servidor remoto.
- O desenvolvedor abre uma "Solicitação de Integração".
- Outros membros revisam o código (*Code Review*).
- A plataforma realiza o merge automaticamente.

Merge Local (CLI)

- Usado em equipes pequenas ou repositórios locais.
- O próprio desenvolvedor faz o *merge* na sua máquina.
- A equipe confia totalmente nos testes e na palavra do desenvolvedor que está integrando.

8. Merge Local da Feature

Se a equipe optar pelo merge via terminal local:

```
# 1. Volte para a develop
git checkout develop

# 2. Atualize com o servidor (evita conflitos pre-merge)
git pull origin develop

# 3. Faça o merge usando a flag --no-ff obrigatoriamente
git merge --no-ff feature/relatorio-vendas

# 4. Apague a branch de feature (limpeza do historico)
git branch -d feature/relatorio-vendas
```

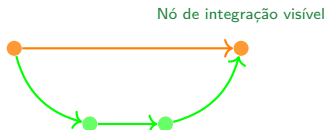
9. Por que usar `-no-ff`?

A flag `-no-ff` (*No Fast-Forward*) força a criação de um **commit de merge**. Sem ela, o Git pode aplicar um *Fast-Forward*, fazendo a feature parecer que foi desenvolvida direto na `develop`.

Fast-Forward (`-ff`)



No Fast-Forward (`-no-ff`)



10. Preparação de Releases

- Quando a `develop` atinge um estado estável contendo as features planejadas, cria-se uma branch de **release**.
- **Congelamento de Features:** A branch de `release` sofre um congelamento parcial. **Não são mais aceitas novas funcionalidades!**
- O que é permitido na branch `release`?
 - Correção de pequenos bugs encontrados pela equipe de QA.
 - Geração de documentação (*Changelogs*).
 - Atualização dos números de versão do sistema.

11. Criando a Release

A branch de release nasce a partir da develop.

```
# 1. Crie a release a partir da develop
git checkout -b release/1.2.0 develop

# 2. Ajustes de meta-dados (ex: mudança de versao)
git commit -am "Bump version to 1.2.0"

# 3. Arrumando bugs de ultima hora durante homologacao
git commit -am "Corrige responsividade no mobile"
```

12. Finalizando a Release

A finalização da release é um **processo duplo**. Ela deve ir para produção (main) e voltar para a develop (para que as correções de bugs não sejam perdidas).

```
# 1. Merge na main e criação da TAG
git checkout main
git merge --no-ff release/1.2.0
git tag -a v1.2.0 -m "Versão Estável 1.2.0"

# 2. Merge de volta na develop
git checkout develop
git merge --no-ff release/1.2.0

# 3. Apagar a branch de release
git branch -d release/1.2.0
```

13. Correções Emergenciais (`hotfix`)

- E se um bug catastrófico for descoberto na produção? O sistema de pagamentos caiu!
- Não podemos usar a `develop`, pois ela já possui recursos pela metade que não podem ir para a produção.
- O `hotfix` existe exatamente para isso: atuar de forma semelhante à `release`, mas iniciando **DIRETAMENTE** da `main` para isolar a correção.

14. O Fluxo do Hotfix

Nasce na main, e retorna para main e develop.

```
# Nasce do estado exato da producao (main)
git checkout -b hotfix/1.2.1 main

# Commitando a correcao do bug
git commit -am "Corrige API de pagamentos (Emergencia)"

# Fechando e mandando para main (com nova TAG minor)
git checkout main
git merge --no-ff hotfix/1.2.1
git tag -a v1.2.1 -m "Hotfix Pagamento"

# Garantindo que a correcao exista na develop
git checkout develop
git merge --no-ff hotfix/1.2.1
```

15. Versionamento com Tags

- Tags funcionam como fotografias inalteráveis (snapshots) do projeto.
- **Automação:** Ferramentas de *Continuous Deployment* (CD) modernas frequentemente monitoram o repositório. Se uma Tag chamada `v2.0.1` for criada, o sistema envia o código automaticamente para produção.
- **Semântica:**
 - Lançamentos regulares aumentam o dígito intermediário (`v1.3.0`).
 - Correções de `hotfix` aumentam o último dígito (`v1.3.1`).